SAP B1 AP Invoice Integration - Concept & Recommendations

Date: 2025-01-27

Last Updated: 2025-01-27
Status: Concept & Recommendations - Ready for Review

Executive Summary

This document outlines the concept, architecture, and recommendations for integrating Laravel Invoice creation with SAP B1 Accounts Payable (AP) Invoice module. The integration will allow invoices created in the DDS system to be manually posted to SAP B1 for accounting and payment processing.

Key Context:

- **AP Invoices follow SAP B1 Purchase flow** (different from AR Invoice Sales flow)
- **Most invoices are based on SAP B1 GRPO** (Goods Receipt PO) - these invoices reference existing GRPO documents
- **Some invoices are Service type** - similar to AR Invoice service pattern, created without GRPO reference
- **Document numbering**: SAP B1 auto-generates `DocNum` (not using DDS invoice number)
- **Sync method**: Manual only (user-triggered, no automatic sync on status change)
- **Bulk operations**: Deferred to future phases

Key Design Decisions ☑

Based on requirements clarification:

1. **Invoice Types**:
   - **GRPO-Based** (most common): Created from SAP B1 Goods Receipt PO, reference GRPO via `po_no`
   - **Service Type**: Created without GRPO, use service item code (similar to AR Invoice pattern)
2. **Document Numbering**:
   - ☑ SAP B1 auto-generates `DocNum` - do not send in payload
   - Store returned `DocNum` in `sap_doc_num` field
3. **Project/Cost Center Mapping**:
   - ☑ Use existing `Project::sap_code` → SAP `ProjectCode`
   - ☑ Use existing `Department::sap_code` → SAP `CostingCode`
   - Reference: Projects-Departments Implementation Guide
4. **Implementation Pattern**:
   - ☑ Follow AR Invoice integration patterns (see AR Invoice Guide)
   - ☑ Use Builder service pattern (`SapApInvoicePayloadBuilder`)
   - ☑ Manual sync only (no automatic triggers)
   - ☑ Queue-based async processing
5. **GRPO Integration**:

- **Phase 1**: Map `po_no` to SAP `Reference1` (simple reference)
- **Phase 2**: Query SAP GRPO and map line items (full integration)

Current State Analysis

☑ What's Already Implemented

1. **Basic Infrastructure**
   - `CreateSapApInvoiceJob` - Queue job for async processing
   - `SapService::createApInvoice()` - Service layer method
   - Database fields: `sap_status`, `sap_doc_num`, `sap_error_message`, `sap_last_attempted_at`
   - `sap_logs` table for audit trail
   - Route: `POST /invoices/{invoice}/sap-sync`
   - Controller method: `InvoiceController::sapSync()`
2. **Session Management**
   - Cookie-based authentication (via Guzzle CookieJar)
   - Automatic session handling
   - Re-login on 401 errors
3. **Basic Payload Structure**
   - Minimal payload with CardCode, DocDate, DocDueDate, Comments
   - Single line item with Quantity=1, UnitPrice=amount, TaxCode='EXEMPT'

⚠ What Needs Enhancement

1. **Incomplete Implementation**
   - Job has incomplete code (lines 42-44)
   - Payload mapping is too simplistic
   - Missing field mappings (PO number, projects, tax handling)
   - No validation before sync
   - Limited error handling
2. **Missing Features**
   - Multi-line item support
   - Tax code determination logic
   - Project/Cost Center mapping
   - Currency handling
   - PO reference linking
   - Document number validation
   - Pre-sync validation UI
   - Status workflow management

Architecture Concept

1. Integration Flow

```
┌─────────────────────────────────────────────────────────────┐
│ User Action: Click "Send to SAP"                            │
└─────────────────────────────────────────────────────────────┘
                          ↓
┌─────────────────────────────────────────────────────────────┐
│ InvoiceController::sapSync()                                │
│ - Validates invoice status                                  │
│ - Checks prerequisites (supplier SAP code, etc.)            │
│ - Updates sap_status to 'pending'                           │
└─────────────────────────────────────────────────────────────┘
                          ↓
┌─────────────────────────────────────────────────────────────┐
│ CreateSapApInvoiceJob (Queued)                              │
│ - Builds SAP payload                                        │
│ - Calls SapService::createApInvoice()                       │
│ - Handles response/errors                                   │
│ - Updates invoice status                                    │
│ - Logs to sap_logs                                          │
└─────────────────────────────────────────────────────────────┘
                          ↓
┌─────────────────────────────────────────────────────────────┐
│ SapService::createApInvoice()                               │
│ - Ensures valid session                                     │
│ - POST to SAP /b1s/v1/Invoices                              │
│ - Returns SAP response                                      │
└─────────────────────────────────────────────────────────────┘
                          ↓
┌─────────────────────────────────────────────────────────────┐
│ SAP B1 Service Layer                                        │
│ - Validates payload                                         │
│ - Creates AP Invoice                                        │
│ - Returns DocEntry, DocNum                                  │
└─────────────────────────────────────────────────────────────┘
```

2. Data Mapping Strategy

Invoice Types & Sources

Type 1: GRPO-Based Invoices (Most Common)

- Created from SAP B1 Goods Receipt PO (GRPO) documents
- Reference existing GRPO via `po_no` field
- Link to GRPO document in SAP for line-item matching
- May need to query SAP GRPO to get line items

Type 2: Service Invoices

- Created without GRPO reference
- Similar pattern to AR Invoice service type (see AR Invoice Guide)
- Use service item code (configurable)
- Single or multi-line items based on invoice details

Laravel Invoice → SAP AP Invoice Mapping

Laravel Field	SAP Field	Mapping Logic	Required	Notes
`supplier.sap_code`	`CardCode`	Direct mapping	☑ Yes	Must exist in SAP as supplier (CardType='S' or 'C')
`invoice_date`	`DocDate`	Format: Y-m-d	☑ Yes	Invoice document date
`payment_date`	`DocDueDate`	Format: Y-m-d, fallback: +30 days	☑ Yes	Payment due date
`invoice_number`	`NumAtCard`	Supplier's invoice number	⚠ Optional	External invoice number from supplier
`faktur_no`	`U_FakturNo` or custom	Faktur Pajak number	⚠ Optional	If custom field exists in SAP
`po_no`	`Reference1` or `U_PONo`	PO/GRPO reference	⚠ Optional	**Critical for GRPO-based invoices**
`amount`	`DocumentLines[].LineTotal`	Per line item	☑ Yes	Total amount per line
`currency`	`DocCurrency`	Currency code (IDR, USD, etc.)	☑ Yes	Must match SAP currency
`invoice_project`	`ProjectCode`	Project code from mapping	⚠ Optional	Use `Project::sap_code` mapping (see Projects Guide)
`payment_project`	`U_PaymentProject`	Payment project	⚠ Optional	Custom field if needed
`cur_loc`	`CostingCode`	Department/Cost Center	⚠ Optional	Map via `Department::sap_code` (see Projects Guide)
`remarks`	`Comments`	Invoice remarks	⚠ Optional	Additional notes
`id`	`U_DDSInvoiceId`	For reconciliation	⚠ Optional	Custom field to link back to DDS

Note: `DocNum` is **auto-generated by SAP B1** - do not send it in payload. SAP returns it in response.

Line Items Strategy

For GRPO-Based Invoices:

- **Option A**: Query SAP GRPO and map line items from GRPO
  - Query SAP for GRPO by `po_no`
  - Map GRPO line items to AP Invoice line items
  - Preserve item codes, quantities, prices from GRPO
  - ☑ Most accurate, matches SAP GRPO exactly

- **Option B**: Single line item with total amount
  - Simple approach, works if GRPO details not needed
  - ⚠ Less accurate, may not match GRPO line-by-line

For Service Invoices:

- Use service item code (from config, similar to AR Invoice pattern)
- Single line item: Quantity=1, UnitPrice=amount
- Or multi-line if invoice has multiple service items

Recommendation:

- **Phase 1**: Start with **Option B** (single line) for both types
- **Phase 2**: Implement **Option A** (GRPO line-item mapping) for GRPO-based invoices
- **Phase 3**: Add multi-line support for Service invoices

3. Status Workflow

```
┌─────────┐
│  open   │ → User creates invoice
└─────────┘
    ↓
┌─────────┐
│ verify  │ → Invoice verified
└─────────┘
    ↓
┌─────────┐
│  sap    │ → Ready for SAP sync
└─────────┘
    ↓
┌─────────┐     ┌─────────┐
│ pending │ → │  posted  │ → Success
└─────────┘     └─────────┘
    ↓
┌─────────┐
│ failed  │ → Error occurred
└─────────┘
```

Status Fields:

- `status`: Business status (open, verify, sap, close, cancel)
- `sap_status`: SAP sync status (null, pending, posted, failed)

Rules:

- Only invoices with `status = 'sap'` can be synced
- **Sync is manual only** - user must click "Send to SAP" button
- `sap_status` transitions: null → pending → posted/failed
- Failed invoices can be retried (reset to pending)
- **DocNum is auto-generated by SAP** - stored in `sap_doc_num` after successful sync

Recommendations

1. Payload Builder Service (High Priority)

Create: `app/Services/SapApInvoicePayloadBuilder.php`

Purpose: Centralize SAP payload construction logic (similar pattern to `SapArInvoiceBuilder`)

Reference: Follow patterns from AR Invoice Builder but adapt for AP Invoice (Purchase flow)

Benefits:

- Separation of concerns
- Easier testing
- Reusable for different invoice types (GRPO-based vs Service)
- Easier to extend for multi-line items
- Consistent with AR Invoice implementation pattern

Key Methods:

```
public function build(Invoice $invoice): array
public function getPreviewData(): array  // For future preview page
public function validate(): array  // Returns validation errors
protected function mapSupplier(Invoice $invoice): string
protected function mapDates(Invoice $invoice): array
protected function mapLineItems(Invoice $invoice): array
protected function mapProjects(Invoice $invoice): array  // Use Project::sap_code
mapping
protected function mapCostCenter(Invoice $invoice): ?string  // Use Department::sap_code
mapping
protected function determineTaxCode(Invoice $invoice): string
protected function isGrpoBased(Invoice $invoice): bool  // Check if invoice has po_no
protected function buildGrpoLineItems(Invoice $invoice): array  // Query SAP GRPO if
needed
protected function buildServiceLineItems(Invoice $invoice): array  // Build service line
items
```

Design Pattern: Similar to `SapArInvoiceBuilder`:

- Constructor takes `Invoice` model
- `build()` returns complete SAP payload
- `validate()` checks prerequisites
- `getPreviewData()` for future UI preview (like AR Invoice preview page)

2. Enhanced Validation (High Priority)

Pre-Sync Validation Checklist:

- ☑ Invoice status = 'sap'
- ☑ Supplier has `sap_code`
- ☑ Supplier exists in SAP (CardType = 'S' or 'C')
- ☑ Invoice amount > 0
- ☑ Invoice date is valid
- ☑ Currency code is valid SAP currency
- ☑ No duplicate SAP document (if `sap_doc` is set)

- ⚠ Project codes exist in SAP (if provided)
- ⚠ Tax code is valid (if custom tax logic)

Implementation:

- Add `Invoice::canSyncToSap()` method
- Add validation in `InvoiceController::sapSync()`
- Show validation errors in UI before queuing

3. Tax Code Handling (Medium Priority)

Current: Hardcoded `'EXEMPT'`

Recommendations:

- **Option A**: Configuration-based mapping

```php
// config/sap.php
'tax_codes' => [
    'default' => 'EXEMPT',
    'by_invoice_type' => [
        'taxable' => 'VAT11',
        'non_taxable' => 'EXEMPT',
    ],
    'by_currency' => [
        'IDR' => 'VAT11',
        'USD' => 'EXEMPT',
    ],
]
```

- **Option B**: Database-driven mapping
  - Add `tax_code` field to `invoice_types` table
  - Map based on invoice type
- **Option C**: SAP Business Partner default
  - Query SAP for supplier's default tax code
  - Use if available, fallback to config

Recommendation: Start with **Option A** (config-based), evolve to **Option C** if needed.

4. Project/Cost Center Mapping (Medium Priority)

Reference: Follow Projects-Departments Implementation Guide

Solution: Use existing `Project` and `Department` models with SAP code mapping

Project Mapping:

- `Project` model already has `sap_code` field
- Use `Project::where('code', $invoice->invoice_project)->first()->sap_code`
- Map to SAP `ProjectCode` field in document lines
- Validate project exists and has `sap_code` before sync

Cost Center/Department Mapping:

- `Department` model already has `sap_code` field (maps to SAP Profit Center)
- Use `Department::where('location_code', $invoice->cur_loc)->first()->sap_code`
- Map to SAP `CostingCode` field in document lines
- Validate department exists and has `sap_code` before sync

Implementation:

```
// In SapApInvoicePayloadBuilder
protected function mapProjects(Invoice $invoice): array
{
    $projectCode = null;
    if ($invoice->invoice_project) {
        $project = Project::where('code', $invoice->invoice_project)->first();
        $projectCode = $project?->sap_code;
    }

    $costCenterCode = null;
    if ($invoice->cur_loc) {
        $department = Department::where('location_code', $invoice->cur_loc)->first();
        $costCenterCode = $department?->sap_code;
    }

    return [
        'project_code' => $projectCode,
        'costing_code' => $costCenterCode,
    ];
}
```

Validation: Add to pre-sync validation:

- If `invoice_project` provided, verify `Project` exists and has `sap_code`
- If `cur_loc` provided, verify `Department` exists and has `sap_code`

5. Currency Handling (Medium Priority)

Current: Direct mapping (assumes currency code matches SAP)

Enhancements:

- Validate currency exists in SAP
- Handle currency conversion if needed
- Support multi-currency invoices (if SAP supports)

Implementation:

```
// In SapService
public function validateCurrency(string $currencyCode): bool
{
    // Query SAP for valid currencies
    // Cache result for performance
}
```

6. GRPO/PO Reference Linking (High Priority for GRPO-Based Invoices)

Context: Most invoices are based on SAP B1 GRPO (Goods Receipt PO)

Current: `po_no` field exists but not mapped

Enhancements:

- **Map `po_no` to SAP `Reference1`** (standard PO reference field)
- **For GRPO-based invoices**: Query SAP GRPO document to get line items
  - Use SAP Service Layer: `GET /GoodsReceiptPOs?$filter=DocNum eq '{po_no}'`
  - Map GRPO line items to AP Invoice line items
  - Preserve item codes, quantities, prices
- **Validate GRPO exists in SAP** (for GRPO-based invoices)
- **Handle Service invoices**: No GRPO reference needed, use service item code

Implementation Priority:

- **Phase 1**: Map `po_no` to `Reference1` only (simple reference)
- **Phase 2**: Query GRPO and map line items (full GRPO integration)

SAP GRPO Query Example:

```php
// In SapService
public function getGrpoByDocNum(string $docNum): ?array
{
    $response = $this->client->get('GoodsReceiptPOs', [
        'query' => [
            '$filter' => "DocNum eq '{$docNum}'",
            '$expand' => 'DocumentLines',
        ]
    ]);

    $data = json_decode($response->getBody()->getContents(), true);
    return $data['value'][0] ?? null;
}
```

7. Error Handling & Retry Logic (High Priority)

Current: Basic retry with exponential backoff

Enhancements:

- **Categorize Errors**:
  - **Transient**: Network issues, session expired → Retry
  - **Validation**: Missing data, invalid format → Don't retry, show error
  - **Business Logic**: Duplicate document, invalid supplier → Don't retry, show error
- **Error Messages**:
  - Parse SAP error responses
  - Show user-friendly messages

  - Store technical details in `sap_error_message`

- **Retry Strategy**:

```
// In CreateSapApInvoiceJob
public function shouldRetry(\Exception $e): bool
{
    // Don't retry validation errors
    if ($e instanceof SapValidationException) {
        return false;
    }

    // Retry transient errors
    if ($e instanceof RequestException && $e->getCode() >= 500) {
        return true;
    }

    return false;
}
```

8. UI Enhancements (Medium Priority)

Reference: Follow UI patterns from AR Invoice Guide

Invoice Show Page:

- Show SAP status badge (similar to AR Invoice status display)
- Show SAP document number (`sap_doc_num` - auto-generated by SAP)
- Show last sync attempt time (`sap_last_attempted_at`)
- Show error message (if failed) (`sap_error_message`)
- **"Send to SAP" button** (only if eligible: `status='sap'` and `sap_status` is null or 'failed')
- **"Retry SAP Sync" button** (if failed)
- Pre-sync validation warnings (show before queuing)
- **Future**: SAP Preview Page (similar to AR Invoice preview) for reviewing payload before submission

Invoice Index Page:

- Add SAP status column (with badge)
- Filter by SAP status (pending, posted, failed, not_sent)
- **Bulk sync action**: Deferred to future phases (per requirements)

UI Pattern: Similar to AR Invoice integration:

- Incomplete/Complete tabs based on `sap_status`
- Preview page for reviewing payload (future enhancement)
- Edit functionality before submission (future enhancement)

9. Reconciliation & Sync Status (Low Priority)

Features:

- Query SAP for invoice by DocNum
- Compare DDS invoice with SAP invoice

- Detect discrepancies
- Sync status updates from SAP

Implementation:

- Add `SapReconciliationService`
- Add command: `php artisan sap:reconcile-invoices`
- Add UI for reconciliation report

10. Testing Strategy (High Priority)

Unit Tests:

- `SapApInvoicePayloadBuilder` - Test payload construction
- `Invoice::canSyncToSap()` - Test validation logic
- `SapService::createApInvoice()` - Mock SAP responses

Integration Tests:

- Test full sync flow with test SAP instance
- Test error scenarios
- Test retry logic

Manual Testing:

- Test with real SAP instance
- Test with various invoice types
- Test error scenarios
- Test retry mechanism

Implementation Priority

Phase 1: Core Functionality (Week 1)

1. ☑ Complete `CreateSapApInvoiceJob` implementation
2. ☑ Create `SapApInvoicePayloadBuilder` service
3. ☑ Add pre-sync validation
4. ☑ Enhance error handling
5. ☑ Fix incomplete code

Phase 2: Field Mapping (Week 2)

1. ☑ Map PO number
2. ☑ Map project codes (if mapping exists)
3. ☑ Map invoice number to NumAtCard
4. ☑ Improve tax code logic
5. ☑ Add currency validation

Phase 3: UI & UX (Week 3)

1. ☑ Add SAP status to invoice show page
2. ☑ Add sync button with validation
3. ☑ Show sync errors in UI
4. ☑ Add retry functionality

5. ☑ Add SAP status filter to index

Phase 4: Advanced Features (Future)

1. ⚠ Multi-line item support
2. ⚠ Project mapping UI
3. ⚠ Reconciliation service
4. ⚠ Bulk sync
5. ⚠ Sync status polling

Technical Considerations

1. Session Management

- **Current**: Each job creates new session
- **Recommendation**: Use singleton `SapService` to reuse sessions
- **Impact**: Reduces session count, improves performance

2. Queue Configuration

- **Current**: Default queue
- **Recommendation**: Use dedicated `sap` queue
- **Impact**: Better isolation, easier monitoring

3. Database Transactions

- **Current**: Transaction wraps status update and logging
- **Recommendation**: Keep as-is
- **Note**: SAP API call happens outside transaction (correct)

4. Logging

- **Current**: Logs to `sap_logs` table and `sap` channel
- **Recommendation**: Keep as-is, add more context
- **Enhancement**: Log payload size, response time

5. Performance

- **Current**: Synchronous API call in job
- **Recommendation**: Keep as-is (SAP API is fast)
- **Future**: Consider batch sync if volume increases

Risk Assessment

High Risk

- **SAP API Changes**: SAP B1 Service Layer may change
  - **Mitigation**: Version API calls, add feature flags
- **Data Mismatch**: DDS data doesn't match SAP requirements
  - **Mitigation**: Comprehensive validation, clear error messages

Medium Risk

- **Session Limits**: Too many concurrent sessions
  - **Mitigation**: Use singleton `SapService`, monitor session count
- **Duplicate Documents**: Same invoice synced twice
  - **Mitigation**: Check `sap_status` before sync, validate `sap_doc` uniqueness

Low Risk

- **Network Issues**: Temporary SAP unavailability
  - **Mitigation**: Retry logic, queue persistence

Success Criteria

Functional Requirements

- ☑ Invoices can be synced to SAP with correct data
- ☑ Validation prevents invalid syncs
- ☑ Errors are clearly communicated
- ☑ Status is accurately tracked
- ☑ Audit trail is complete

Non-Functional Requirements

- ☑ Sync completes within 5 seconds (excluding queue time)
- ☑ 99% success rate (excluding validation errors)
- ☑ Errors are logged and traceable
- ☑ UI is intuitive and informative

Next Steps

1. **Review this document** with stakeholders
2. **Prioritize features** based on business needs
3. **Create detailed action plan** for Phase 1
4. **Set up test SAP instance** (if not available)
5. **Begin implementation** starting with Phase 1

Resolved Questions ☑

1. **Tax Handling**: ☑ Configuration-based mapping (similar to AR Invoice pattern)
   - Use config file with default tax codes
   - Map by invoice type or currency if needed
   - Reference: AR Invoice uses config-based approach
2. **Project Mapping**: ☑ Use existing `Project` and `Department` models
   - `Project::sap_code` maps to SAP `ProjectCode`
   - `Department::sap_code` maps to SAP `CostingCode`
   - Reference: Projects-Departments Implementation Guide
3. **Multi-Line Items**: ☑ Deferred to later phases
   - Phase 1: Single line item

    - Phase 2: GRPO line-item mapping
    - Phase 3: Multi-line support for Service invoices

4. **Custom Fields**: ⚠ To be determined during implementation
    - May use `U_DDSInvoiceId` for reconciliation
    - May use `U_FakturNo` if faktur tracking needed
    - Will be confirmed with SAP B1 configuration
5. **Document Numbering**: ☑ **SAP auto-generates DocNum**
    - Do not send `DocNum` in payload
    - SAP returns `DocNum` in response
    - Store in `sap_doc_num` field
6. **Sync Timing**: ☑ **Manual only**
    - User clicks "Send to SAP" button
    - No automatic sync on status change
    - Similar to AR Invoice manual submission pattern
7. **Bulk Operations**: ☑ **Deferred to future phases**
    - Phase 1-3: One-by-one manual sync
    - Future: Bulk sync functionality

References

Primary References

- **SAP B1 AR Invoice Developer Guide** - Reference implementation for similar invoice integration patterns
- **Projects-Departments Implementation Guide** - Project and cost center mapping patterns
- **SAP B1 Session Management** - Cookie-based authentication and session handling

Additional References

- SAP B1 Service Layer Documentation
- Current Implementation
- ITO Sync Implementation

Key Differences: AP Invoice vs AR Invoice

Aspect	AR Invoice (Sales)	AP Invoice (Purchase)
Document Type	Sales Invoice	Purchase Invoice
CardCode	Customer (CardType='C')	Supplier (CardType='S' or 'C')
Source Document	Faktur (Sales)	GRPO (Goods Receipt PO) or Service
Account	AR Account (Debit)	AP Account (Credit)
Line Items	Service items	GRPO items or Service items
WTax	2% of DPP (Withholding Tax)	May vary by supplier
Journal Entry	Creates JE separately	May create JE separately (future)

Aspect	AR Invoice (Sales)	AP Invoice (Purchase)
Pattern	Dual document (AR + JE)	Single document (AP Invoice) initially

Note: AP Invoice integration should follow similar patterns to AR Invoice but adapted for Purchase flow.